製造業BPO詳細設計

スコープ: 中小製造業（10-300名）、金属加工・機械・食品・化学・電子部品
実装時期: Phase 2+（建設業BPOのPMF確認後）
設計の正: この文書 + `d_00_BPOアーキテクチャ設計.md`

---

0. エグゼクティブサマリー

なぜ製造業BPOか

```
製造業の中小企業（10-300名）: 約5.2万社（経産省 工業統計調査）
DX意欲あり層: 約15-20% = 約8,000-10,000社

製造業の中小企業は：
  ・見積に時間がかかりすぎて受注機会を逸失している
  ・生産計画がExcel職人の頭の中にしかない
  ・品質データは取っているが分析できていない
  ・在庫が多すぎるか足りないかの二択
  ・ISOの文書管理だけで専任1人分の工数
 → 全部AIで代行できる = BPOコア¥25万 + 追加モジュール¥10万/個の価値
```

8モジュール全体像

```
┌──────────────────────────────────────────────────────────────┐
│              製造業BPO（BPOコア¥250,000 + 追加モジュール¥100,000/個）                  │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│  ★ Phase A: 最優先                                           │
│  ┌──────────────────┐  ┌──────────────────────────┐         │
│  │ ① 見積AI          │  │ ② 生産計画AI              │        │
│  │ 図面→工程→見積書   │  │ 受注→山積み→工程表        │        │
│  │ ¥10-30万/月の価値  │  │ ¥5-10万/月の価値          │        │
│  └──────────────────┘  └──────────────────────────┘         │
│                                                              │
│  Phase B:                                                    │
│  ┌──────────────────┐  ┌──────────────────────────┐         │
│  │ ③ 品質管理        │  │ ④ 在庫最適化              │        │
│  │ SPC+不良予測      │  │ 需要予測→発注提案         │        │
│  └──────────────────┘  └──────────────────────────┘         │
│                                                              │
│  Phase C:                                                    │
│  ┌──────────────────┐  ┌──────────────────────────┐         │
│  │ ⑤ SOP管理         │  │ ⑥ 設備保全                │        │
│  │ 手順書AI作成・改訂 │  │ 保全計画+故障予測         │        │
│  └──────────────────┘  └──────────────────────────┘         │
│                                                              │
│  Phase D:                                                    │
│  ┌──────────────────┐  ┌──────────────────────────┐         │
│  │ ⑦ 仕入管理        │  │ ⑧ ISO文書管理             │        │
│  │ BOM連動→発注→支払 │  │ 9001/14001文書体系        │        │
│  └──────────────────┘  └──────────────────────────┘         │
└──────────────────────────────────────────────────────────────┘
```

1. 見積AI（★キラーフィーチャー）

1.1 問題の本質

```
製造業の見積の現状:
  1. 顧客から図面（PDF/DXF/STEP）+ 仕様書を受け取る
  2. 図面を読んで加工工程を推定する
     - 材料切断 → 旋盤加工 → MC加工 → 研磨 → 表面処理 → 検査
  3. 各工程の工数を算出する
     - 段取り時間 + 加工時間 × 数量
  4. チャージレート（設備の時間単価）を掛ける
     - 汎用旋盤: ¥4,000/h、CNC旋盤: ¥6,000/h、MC: ¥8,000/h 等
  5. 材料費・表面処理費・外注費・管理費を加算する
  6. 見積書を作成して顧客に回答する

  → 熟練者が1件30分〜数時間。月100件以上の引合いに対応
  → 回答が遅い = 受注機会の逸失（顧客は3社見積で最速回答に発注しがち）
  → 見積精度が低い = 赤字受注 or 高すぎて失注
  → 熟練見積担当の属人化。退職リスク
```

1.2 見積AIの3段階アプローチ

```
■ Level 1: 工程推定AI（最初に実装）
  入力: 図面PDF/画像 + 仕様（材質、数量、表面処理）
  処理: Document AI + LLM で形状・寸法を認識 → 加工工程を推定
  出力: 推定工程表（工程名、設備、推定工数）

  ユーザーがチャージレート・工数を確認/修正 → 見積書を自動生成

  価値: 見積作成時間の50%削減
  制約: 工数の精度は初期60-70%（人間の確認必須）

■ Level 2: 実績学習型見積（3ヶ月後）
  入力: Level 1 + 過去の見積実績 + 実際の加工実績
  処理:
    ① 類似部品の過去見積を検索（形状・材質・サイズで類似度判定）
    ② 過去実績から工数を推定（見積工数 vs 実績工数の補正）
    ③ 受注率分析（価格帯別の受注率 → 最適価格帯を提案）
  出力: 高精度な見積 + 受注率予測

  価値: 見積作成時間の80%削減 + 受注率向上

■ Level 3: ネットワーク効果（Phase 3+）
  - N社の匿名化見積データから市場単価を推定
  - 「同業他社のMC加工チャージレート: ¥7,500-9,000/h」
  - 業種別の利益率ベンチマーク
```

1.3 データモデル

```
-- 見積プロジェクト
CREATE TABLE mfg_quotes (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  quote_number TEXT NOT NULL,              -- 見積番号（自動採番）
  customer_name TEXT NOT NULL,             -- 顧客名
  project_name TEXT,                       -- 案件名
  quantity INTEGER NOT NULL DEFAULT 1,     -- 数量
  material TEXT,                           -- 材質（SS400, SUS304, A5052 等）
  surface_treatment TEXT,                  -- 表面処理（メッキ、アルマイト、塗装等）
  delivery_date DATE,                      -- 希望納期
  total_amount BIGINT,                     -- 見積金額（円）
  profit_margin DECIMAL(5,2),             -- 利益率（%）
  status TEXT DEFAULT 'draft',             -- draft / sent / won / lost / expired
  won_amount BIGINT,                       -- 受注金額（値引き後）
  lost_reason TEXT,                        -- 失注理由
  valid_until DATE,                        -- 見積有効期限
  file_url TEXT,                           -- 見積書PDF URL
  metadata JSONB DEFAULT '{}',
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

-- 見積明細（工程別）
CREATE TABLE mfg_quote_items (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  quote_id UUID NOT NULL REFERENCES mfg_quotes(id) ON DELETE CASCADE,
  company_id UUID NOT NULL REFERENCES companies(id),
  sort_order INTEGER NOT NULL,
  process_name TEXT NOT NULL,              -- 工程名（材料切断、旋盤、MC、研磨等）
  equipment TEXT,                          -- 使用設備（CNC旋盤、マシニングセンタ等）
  setup_time_min DECIMAL(8,1),            -- 段取り時間（分）
  cycle_time_min DECIMAL(8,1),            -- サイクルタイム（分/個）
  total_time_min DECIMAL(10,1),           -- 合計時間 = 段取り + (サイクル × 数量)
  charge_rate DECIMAL(10,0),              -- チャージレート（円/時間）
  process_cost BIGINT,                     -- 加工費 = 合計時間/60 × チャージレート
  material_cost BIGINT,                    -- 材料費（材料工程のみ）
  outsource_cost BIGINT,                   -- 外注費（外注工程のみ）
  cost_source TEXT,                        -- manual / past_record / ai_estimated
  confidence DECIMAL(3,2),                 -- 推定信頼度
  notes TEXT,
  created_at TIMESTAMPTZ DEFAULT now()
);

-- チャージレートマスタ
CREATE TABLE mfg_charge_rates (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  equipment_name TEXT NOT NULL,            -- 設備名
```

```
  equipment_type TEXT NOT NULL,             -- 設備種別（lathe / machining_center /
grinder / press 等）
  charge_rate DECIMAL(10,0) NOT NULL,      -- チャージレート（円/時間）
  setup_time_default DECIMAL(8,1),         -- デフォルト段取り時間（分）
  notes TEXT,
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

-- 材料単価マスタ
CREATE TABLE mfg_material_prices (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  material_code TEXT NOT NULL,              -- 材質コード（SS400, SUS304 等）
  material_name TEXT NOT NULL,              -- 材質名
  form TEXT NOT NULL,                       -- 形状（丸棒、板、パイプ等）
  size_spec TEXT,                           -- サイズ仕様（φ50, t10 等）
  unit TEXT NOT NULL,                       -- 単位（kg, m, 枚 等）
  unit_price DECIMAL(10,0) NOT NULL,       -- 単価
  supplier TEXT,                            -- 仕入先
  updated_at TIMESTAMPTZ DEFAULT now()
);

-- RLS
ALTER TABLE mfg_quotes ENABLE ROW LEVEL SECURITY;
ALTER TABLE mfg_quote_items ENABLE ROW LEVEL SECURITY;
ALTER TABLE mfg_charge_rates ENABLE ROW LEVEL SECURITY;
ALTER TABLE mfg_material_prices ENABLE ROW LEVEL SECURITY;
```

1.4 見積AIパイプライン

```
# workers/bpo/manufacturing/quoting.py

class QuotingPipeline:
    """
    製造業 見積AIパイプライン

    Step 1: 図面の取り込み・解析
    Step 2: 加工工程の推定
    Step 3: 工数・コストの算出
    Step 4: 見積書生成
    """

    async def analyze_drawing(
        self,
        files: list[UploadFile],
        material: str,
        quantity: int,
    ) -> DrawingAnalysis:
        """
        図面を解析して形状・寸法・加工要素を抽出

        対応形式:
        - 図面PDF/画像 → Document AI + LLM
        - 3D CAD (STEP/IGES) → 形状解析 (Phase 2+)
        - Excel仕様書 → 構造化抽出
        """

    async def estimate_processes(
        self,
        analysis: DrawingAnalysis,
        material: str,
        surface_treatment: str | None,
    ) -> list[ProcessEstimate]:
        """
        加工工程を推定

        LLMプロンプト:
         「以下の部品の加工工程を推定してください。
          形状: {shape_description}
          材質: {material}
          寸法: {dimensions}
          表面処理: {surface_treatment}
          公差: {tolerances}

          出力: 工程順に [{process_name, equipment, setup_time_min, cycle_time_min,
notes}]」

        過去実績がある場合:
        → 類似部品の実績工数で補正
```

```
        """

    async def calculate_costs(
        self,
        quote_id: str,
        processes: list[ProcessEstimate],
        quantity: int,
    ) -> QuoteCostBreakdown:
        """
        コスト計算

        見積金額 = 材料費 + Σ(加工費) + 表面処理費 + 外注費 + 管理費
        加工費 = (段取り時間 + サイクルタイム × 数量) / 60 × チャージレート
        管理費 = (材料費 + 加工費 + 表面処理 + 外注費) × 管理費率
        """

    async def generate_quote_document(
        self,
        quote_id: str,
        format: str = "pdf",
    ) -> bytes:
        """見積書PDF/Excel生成"""

    async def analyze_win_rate(
        self,
        quote_id: str,
    ) -> WinRateAnalysis:
        """
        受注率分析
        - この価格帯での過去の受注率
        - 「¥X万なら受注率70%、¥Y万なら50%」
        - 利益率と受注率のバランス提案
        """

    async def learn_from_actual(
        self,
        quote_id: str,
        actual_times: list[dict],
    ) -> None:
        """
        実績からの学習
        - 見積工数 vs 実績工数の差分を記録
        - 次回以降の工数推定精度を向上
        """
```

1.5 UI設計

```
■ 見積一覧: /bpo/manufacturing/quotes
  - 見積一覧（番号、顧客、金額、状態、受注率）
  - 新規作成 / CSV一括取り込み
  - フィルター（状態、顧客、期間）

■ 見積作成: /bpo/manufacturing/quotes/new
  Step 1: 基本情報（顧客、案件名、数量、材質、表面処理、納期）
  Step 2: 図面アップロード → AI解析中...
  Step 3: 工程確認・編集（AI推定結果をテーブル表示、行ごと編集可）
  Step 4: コスト確認（材料費、加工費、管理費の内訳表示）
  Step 5: 見積書出力（PDF/Excel）

■ チャージレート管理: /bpo/manufacturing/charge-rates
  - 設備別チャージレート一覧
  - 設備追加・編集

■ 材料単価管理: /bpo/manufacturing/materials
  - 材質別単価一覧
  - 仕入先別比較
```

---

2. 生産計画AI

2.1 問題の本質

```
中小製造業の生産計画:
  - ベテラン工場長がExcelとホワイトボードで管理
  - 「この設備は来週火曜まで空かない」が頭の中にしかない
  - 急な飛び込み受注で計画が崩壊 → 納期遅延
  - 設備稼働率が見えない → 投資判断ができない
```

2.2 データモデル

```
-- 受注（生産指示の元）
CREATE TABLE mfg_orders (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  order_number TEXT NOT NULL,
  customer_name TEXT NOT NULL,
  product_name TEXT NOT NULL,
  quantity INTEGER NOT NULL,
  due_date DATE NOT NULL,                   -- 納期
  priority TEXT DEFAULT 'normal',           -- urgent / high / normal / low
  quote_id UUID REFERENCES mfg_quotes(id),
  status TEXT DEFAULT 'pending',            -- pending / planned / in_progress /
completed / shipped
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

-- 設備マスタ
CREATE TABLE mfg_equipment (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  name TEXT NOT NULL,                       -- 設備名
  equipment_type TEXT NOT NULL,             -- lathe / mc / grinder / press / welder 等
  capacity_hours_per_day DECIMAL(4,1) DEFAULT 8.0,  -- 1日の稼働可能時間
  status TEXT DEFAULT 'active',             -- active / maintenance / inactive
  notes TEXT,
  created_at TIMESTAMPTZ DEFAULT now()
);

-- 生産スケジュール（設備×日×オーダー）
CREATE TABLE mfg_schedules (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  order_id UUID NOT NULL REFERENCES mfg_orders(id),
  equipment_id UUID NOT NULL REFERENCES mfg_equipment(id),
  process_name TEXT NOT NULL,
  scheduled_date DATE NOT NULL,
  start_time TIME,
  duration_hours DECIMAL(5,1) NOT NULL,     -- 予定時間
  actual_hours DECIMAL(5,1),                -- 実績時間
  status TEXT DEFAULT 'planned',            -- planned / in_progress / completed /
delayed
  created_at TIMESTAMPTZ DEFAULT now()
);

-- RLS
ALTER TABLE mfg_orders ENABLE ROW LEVEL SECURITY;
```

```
ALTER TABLE mfg_equipment ENABLE ROW LEVEL SECURITY;
ALTER TABLE mfg_schedules ENABLE ROW LEVEL SECURITY;
```

2.3 生産計画エンジン

```
# workers/bpo/manufacturing/production_plan.py

class ProductionPlanner:
    """
    生産計画AI

    山積み: 受注を納期順に並べて設備に積む
    山崩し: 設備キャパを超えた分を前倒し/外注/残業で調整
    """

    async def generate_plan(
        self,
        company_id: str,
        start_date: date,
        end_date: date,
    ) -> ProductionPlan:
        """
        生産計画を自動生成
        1. 未完了の受注を納期順にソート
        2. 各受注の工程を設備に割当（山積み）
        3. キャパ超過を検出（山崩し）
        4. ガントチャートデータを生成
        """

    async def simulate_new_order(
        self,
        order: NewOrderInput,
    ) -> SimulationResult:
        """
        新規受注のシミュレーション
        「この案件を受けたら既存の納期に影響するか？」
        → 影響ありの場合、対策案（残業/外注/納期調整）を提示
        """

    async def get_equipment_utilization(
        self,
        company_id: str,
        period: str,
    ) -> EquipmentUtilization:
        """設備稼働率レポート"""
```

3. 品質管理

3.1 データモデル

```
-- 検査記録
CREATE TABLE mfg_inspections (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  order_id UUID REFERENCES mfg_orders(id),
  product_name TEXT NOT NULL,
  lot_number TEXT,
  inspection_type TEXT NOT NULL,            -- incoming / in_process / final / patrol
  inspector TEXT,
  inspection_date DATE NOT NULL,
  items JSONB NOT NULL,                     -- [{name, spec_min, spec_max, measured,
unit, result}]
  overall_result TEXT NOT NULL,             -- pass / fail / conditional
  defect_count INTEGER DEFAULT 0,
  defect_details JSONB,                     -- [{type, count, description}]
  notes TEXT,
  created_at TIMESTAMPTZ DEFAULT now()
);

-- 不良記録
CREATE TABLE mfg_defects (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  inspection_id UUID REFERENCES mfg_inspections(id),
  order_id UUID REFERENCES mfg_orders(id),
  defect_date DATE NOT NULL,
  product_name TEXT NOT NULL,
  defect_type TEXT NOT NULL,                -- dimensional / surface / material /
assembly
  defect_detail TEXT NOT NULL,
  quantity INTEGER NOT NULL DEFAULT 1,
  cause TEXT,                               -- machine / material / method / man /
measurement
  corrective_action TEXT,
  preventive_action TEXT,
  status TEXT DEFAULT 'open',               -- open / investigating / resolved
  created_at TIMESTAMPTZ DEFAULT now()
);

-- RLS
ALTER TABLE mfg_inspections ENABLE ROW LEVEL SECURITY;
ALTER TABLE mfg_defects ENABLE ROW LEVEL SECURITY;
```

3.2 品質管理エンジン

```
# workers/bpo/manufacturing/quality.py

class QualityManager:
    """
    品質管理AI
    - SPC（統計的工程管理）: Xbar-R管理図、Cp/Cpk計算
    - 不良分析: パレート図、特性要因図データ
    - 不良予測: トレンドから不良発生を予測
    """

    async def calculate_spc(
        self,
        product_name: str,
        measurement_name: str,
        period_days: int = 30,
    ) -> SPCResult:
        """Xbar-R管理図 + Cp/Cpk"""

    async def generate_quality_report(
        self,
        company_id: str,
        year: int,
        month: int,
    ) -> QualityReport:
        """月次品質レポート（不良率、パレート、トレンド）"""

    async def predict_defects(
        self,
        product_name: str,
    ) -> DefectPrediction:
        """不良予測（トレンド分析から）"""
```

4. 在庫最適化

4.1 データモデル

```
-- 在庫マスタ
CREATE TABLE mfg_inventory (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  item_code TEXT NOT NULL,
  item_name TEXT NOT NULL,
  item_type TEXT NOT NULL,                  -- raw_material / wip / finished /
consumable
  unit TEXT NOT NULL,
  current_stock DECIMAL(12,2) NOT NULL DEFAULT 0,
  safety_stock DECIMAL(12,2) DEFAULT 0,     -- 安全在庫
  reorder_point DECIMAL(12,2),              -- 発注点
  reorder_quantity DECIMAL(12,2),           -- 発注量（EOQ）
  lead_time_days INTEGER,                   -- リードタイム
  unit_cost DECIMAL(10,0),                  -- 単価
  location TEXT,                            -- 保管場所
  last_counted_at TIMESTAMPTZ,             -- 最終棚卸日
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now(),
  UNIQUE(company_id, item_code)
);

-- 在庫移動ログ
CREATE TABLE mfg_inventory_transactions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  item_id UUID NOT NULL REFERENCES mfg_inventory(id),
  transaction_type TEXT NOT NULL,            -- receive / issue / adjust / return
  quantity DECIMAL(12,2) NOT NULL,
  reference_type TEXT,                       -- order / purchase / adjustment
  reference_id UUID,
  notes TEXT,
  created_at TIMESTAMPTZ DEFAULT now()
);

-- RLS
ALTER TABLE mfg_inventory ENABLE ROW LEVEL SECURITY;
ALTER TABLE mfg_inventory_transactions ENABLE ROW LEVEL SECURITY;
```

4.2 在庫最適化エンジン

```
# workers/bpo/manufacturing/inventory.py

class InventoryOptimizer:
    """
    在庫最適化AI
    - 需要予測（過去実績 + 受注見込み → 将来消費量推定）
    - 安全在庫計算（需要変動 × リードタイム変動）
    - 発注提案（発注点割れ → 自動通知）
    - 棚卸サポート（差異分析）
    """

    async def suggest_reorder(
        self,
        company_id: str,
    ) -> list[ReorderSuggestion]:
        """発注点割れの品目を検出し、発注提案"""

    async def forecast_demand(
        self,
        item_id: str,
        months_ahead: int = 3,
    ) -> DemandForecast:
        """需要予測（移動平均 + 受注残からの推定）"""

    async def generate_inventory_report(
        self,
        company_id: str,
    ) -> InventoryReport:
        """在庫レポート（金額、回転率、滞留品）"""
```

5. SOP管理

5.1 データモデル

```
-- 作業手順書
CREATE TABLE mfg_sops (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  sop_number TEXT NOT NULL,                -- 文書番号
  title TEXT NOT NULL,
  process_name TEXT NOT NULL,              -- 対象工程
  version INTEGER DEFAULT 1,
  status TEXT DEFAULT 'draft',             -- draft / review / approved / obsolete
  content JSONB NOT NULL,                  -- [{step_number, description, image_url,
caution, tools_needed}]
  approved_by UUID REFERENCES users(id),
  approved_at TIMESTAMPTZ,
  revision_history JSONB DEFAULT '[]',     -- [{version, date, changed_by, reason}]
  next_review_date DATE,
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

ALTER TABLE mfg_sops ENABLE ROW LEVEL SECURITY;
```

5.2 SOP管理エンジン

```
# workers/bpo/manufacturing/sop_manager.py

class SOPManager:
    """
    SOP管理AI
    - 手順書のAI生成（工程名+条件 → 手順書ドラフト）
    - 改訂管理（変更理由の記録、旧版アーカイブ）
    - レビューリマインド（定期見直し）
    - 多言語対応（外国人作業者向け翻訳）
    """

    async def generate_sop(
        self,
        process_name: str,
        conditions: str,
        equipment: str,
    ) -> SOPDraft:
        """LLMで手順書ドラフトを生成"""

    async def check_review_schedule(
        self,
        company_id: str,
    ) -> list[SOPReviewAlert]:
        """レビュー期限が近い手順書を検出"""
```

6. 設備保全

6.1 データモデル

```
-- 保全記録
CREATE TABLE mfg_maintenance_records (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  equipment_id UUID NOT NULL REFERENCES mfg_equipment(id),
  maintenance_type TEXT NOT NULL,            -- daily / weekly / monthly / annual /
breakdown
  maintenance_date DATE NOT NULL,
  description TEXT NOT NULL,
  parts_replaced JSONB,                      -- [{part_name, quantity, cost}]
  downtime_hours DECIMAL(5,1),              -- 停止時間
  cost BIGINT,                               -- 保全費用
  performed_by TEXT,
  next_scheduled DATE,
  notes TEXT,
  created_at TIMESTAMPTZ DEFAULT now()
);

ALTER TABLE mfg_maintenance_records ENABLE ROW LEVEL SECURITY;
```

6.2 設備保全エンジン

```
# workers/bpo/manufacturing/maintenance.py

class MaintenanceManager:
    """
    設備保全AI
    - 保全カレンダー生成（日次/週次/月次/年次）
    - 点検チェックリスト管理
    - 故障予兆検知（故障間隔のトレンド分析）
    - 保全コストレポート
    """

    async def generate_maintenance_calendar(
        self,
        company_id: str,
        month: int,
        year: int,
    ) -> MaintenanceCalendar:
        """保全カレンダー生成"""

    async def predict_failure(
        self,
        equipment_id: str,
    ) -> FailurePrediction:
        """故障予測（MTBF分析）"""
```

7. 仕入管理

7.1 データモデル

```
-- 部品表（BOM）
CREATE TABLE mfg_bom (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  product_code TEXT NOT NULL,
  product_name TEXT NOT NULL,
  items JSONB NOT NULL,                      -- [{item_code, item_name, quantity, unit}]
  version INTEGER DEFAULT 1,
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

-- 発注
CREATE TABLE mfg_purchase_orders (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  po_number TEXT NOT NULL,
  vendor_id UUID NOT NULL REFERENCES bpo_vendors(id),
  order_date DATE NOT NULL,
  expected_date DATE,                        -- 希望納期
  items JSONB NOT NULL,                      -- [{item_code, item_name, quantity, unit,
unit_price, amount}]
  total_amount BIGINT NOT NULL,
  status TEXT DEFAULT 'draft',               -- draft / sent / partially_received /
received / cancelled
  received_at TIMESTAMPTZ,
  notes TEXT,
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

-- RLS
ALTER TABLE mfg_bom ENABLE ROW LEVEL SECURITY;
ALTER TABLE mfg_purchase_orders ENABLE ROW LEVEL SECURITY;
```

7.2 仕入管理エンジン

```
# workers/bpo/manufacturing/procurement.py

class ProcurementManager:
    """
    仕入管理
    - BOM展開 → 必要資材の自動算出
    - 発注書の自動生成
    - 検収管理（納品チェック）
    - 仕入先評価（QCD）
    """

    async def explode_bom(
        self,
        product_code: str,
        quantity: int,
    ) -> list[MaterialRequirement]:
        """BOM展開 → 所要量計算"""

    async def generate_purchase_order(
        self,
        requirements: list[MaterialRequirement],
    ) -> PurchaseOrder:
        """発注書自動生成"""
```

---

8. ISO文書管理

8.1 データモデル

```
-- ISO文書
CREATE TABLE mfg_iso_documents (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  document_number TEXT NOT NULL,             -- 文書番号
  title TEXT NOT NULL,
  document_type TEXT NOT NULL,               -- manual / procedure / work_instruction /
record / form
  iso_clause TEXT,                           -- 対応するISO条項（4.1, 7.1.5 等）
  version INTEGER DEFAULT 1,
  status TEXT DEFAULT 'draft',               -- draft / review / approved / obsolete
  content_url TEXT,                          -- 文書ファイルURL
  approved_by UUID REFERENCES users(id),
  approved_at TIMESTAMPTZ,
  next_review_date DATE,
  revision_history JSONB DEFAULT '[]',
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

-- 内部監査
CREATE TABLE mfg_internal_audits (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  audit_date DATE NOT NULL,
  auditor TEXT NOT NULL,
  department TEXT NOT NULL,
  iso_clauses TEXT[] NOT NULL,              -- 監査対象条項
  findings JSONB DEFAULT '[]',              -- [{type: observation/minor/major, clause,
description, corrective_action, due_date, status}]
  overall_result TEXT,                       -- conforming / minor_nc / major_nc
  notes TEXT,
  created_at TIMESTAMPTZ DEFAULT now()
);

-- RLS
ALTER TABLE mfg_iso_documents ENABLE ROW LEVEL SECURITY;
ALTER TABLE mfg_internal_audits ENABLE ROW LEVEL SECURITY;
```

8.2 ISO文書管理エンジン

```
# workers/bpo/manufacturing/iso_docs.py

class ISODocumentManager:
    """
    ISO文書管理AI
    - 文書体系の管理（マニュアル→手順書→作業指示書→記録）
    - 改訂管理（承認フロー付き）
    - 内部監査チェックリスト自動生成
    - 不適合の是正処置管理
    - 審査準備レポート
    """

    async def generate_audit_checklist(
        self,
        department: str,
        iso_standard: str = "9001",
    ) -> AuditChecklist:
        """内部監査チェックリスト生成（LLM + ISO条項）"""

    async def check_document_review(
        self,
        company_id: str,
    ) -> list[DocumentReviewAlert]:
        """レビュー期限が近い文書を検出"""

    async def generate_management_review_input(
        self,
        company_id: str,
        period: str,
    ) -> ManagementReviewReport:
        """マネジメントレビュー用インプットレポート自動生成"""
```

9. ディレクトリ構成

```
workers/bpo/manufacturing/
├── __init__.py
├── quoting.py              # ① 見積AI
├── production_plan.py      # ② 生産計画AI
├── quality.py              # ③ 品質管理
├── inventory.py            # ④ 在庫最適化
├── sop_manager.py          # ⑤ SOP管理
├── maintenance.py          # ⑥ 設備保全
├── procurement.py          # ⑦ 仕入管理
├── iso_docs.py             # ⑧ ISO文書管理
└── models.py               # Pydanticモデル

routers/bpo/manufacturing.py  # 全エンドポイント

frontend/src/app/(authenticated)/bpo/manufacturing/
├── quotes/                 # 見積
├── production/             # 生産計画
├── quality/                # 品質管理
├── inventory/              # 在庫
├── sop/                    # SOP
├── maintenance/            # 設備保全
├── procurement/            # 仕入
└── iso/                    # ISO文書

db/migrations/
└── 008_bpo_manufacturing.sql
```

10. APIエンドポイント

```
# 見積
POST   /api/bpo/manufacturing/quotes                    # 見積作成
GET    /api/bpo/manufacturing/quotes                    # 一覧
GET    /api/bpo/manufacturing/quotes/{id}               # 詳細
PATCH  /api/bpo/manufacturing/quotes/{id}               # 更新
POST   /api/bpo/manufacturing/quotes/{id}/analyze       # 図面AI解析
POST   /api/bpo/manufacturing/quotes/{id}/export        # 見積書出力
GET    /api/bpo/manufacturing/charge-rates              # チャージレート一覧
POST   /api/bpo/manufacturing/charge-rates              # チャージレート登録
GET    /api/bpo/manufacturing/materials                 # 材料単価一覧

# 生産計画
POST   /api/bpo/manufacturing/orders                    # 受注登録
GET    /api/bpo/manufacturing/orders                    # 受注一覧
POST   /api/bpo/manufacturing/schedule/generate         # 生産計画生成
GET    /api/bpo/manufacturing/schedule                  # 生産計画表示
POST   /api/bpo/manufacturing/schedule/simulate         # 新規受注シミュレーション
GET    /api/bpo/manufacturing/equipment                 # 設備一覧
GET    /api/bpo/manufacturing/equipment/utilization      # 稼働率

# 品質管理
POST   /api/bpo/manufacturing/inspections               # 検査記録登録
GET    /api/bpo/manufacturing/inspections               # 検査記録一覧
GET    /api/bpo/manufacturing/quality/spc/{product}     # SPC分析
GET    /api/bpo/manufacturing/quality/report             # 品質レポート
POST   /api/bpo/manufacturing/defects                   # 不良登録

# 在庫
GET    /api/bpo/manufacturing/inventory                 # 在庫一覧
PATCH  /api/bpo/manufacturing/inventory/{id}            # 在庫更新
GET    /api/bpo/manufacturing/inventory/reorder          # 発注提案
GET    /api/bpo/manufacturing/inventory/report           # 在庫レポート

# SOP
POST   /api/bpo/manufacturing/sops                      # SOP作成
GET    /api/bpo/manufacturing/sops                      # SOP一覧
POST   /api/bpo/manufacturing/sops/{id}/generate        # AI生成
GET    /api/bpo/manufacturing/sops/review-alerts         # レビューアラート

# 設備保全
POST   /api/bpo/manufacturing/maintenance               # 保全記録登録
GET    /api/bpo/manufacturing/maintenance/calendar       # 保全カレンダー
GET    /api/bpo/manufacturing/maintenance/prediction/{eq_id}  # 故障予測

# 仕入
POST   /api/bpo/manufacturing/purchase-orders           # 発注書作成
GET    /api/bpo/manufacturing/purchase-orders           # 発注一覧
POST   /api/bpo/manufacturing/bom/explode               # BOM展開
```

```
# ISO
POST   /api/bpo/manufacturing/iso/documents            # 文書登録
GET    /api/bpo/manufacturing/iso/documents            # 文書一覧
POST   /api/bpo/manufacturing/iso/audit-checklist       # 監査チェックリスト生成
POST   /api/bpo/manufacturing/iso/audits               # 監査記録登録
```

---

11. SaaS連携戦略

```
Phase 1: kintone（多くの製造業が使用）→ 受注・在庫データ取得
Phase 2: zaico API → 在庫データ連携
Phase 3: freee/MF → 会計連携（原価計算）
Phase 4: techs 等の生産管理SaaS → API確認後に連携
```

---

12. 並列開発プラン

```
Phase A（2週間）: 見積AI + 生産計画 — 4エージェント
Phase B（1週間）: 品質管理 + 在庫 — 3エージェント
Phase C（1週間）: SOP + 設備保全 — 3エージェント
Phase D（1週間）: 仕入 + ISO — 3エージェント
```

---

13. リスクと対策

リスク	対策
図面解析の精度	最初はExcel仕様書取り込みを優先。図面OCRは補助
チャージレートの設定	ユーザーが自社設備ごとに設定。業界平均値をデフォルト提供
生産計画の複雑性	MVPは単純なガントチャート。制約条件の最適化はPhase 2+
SPC計算の誤り	統計ライブラリ（scipy）使用。手計算結果と照合テスト

---

14. KPI・成功基準

完了基準

- [ ] 見積作成時間が50%以上短縮
- [ ] 生産計画がガントチャートで可視化
- [ ] 品質レポートが自動生成
- [ ] パイロット3社が実業務で使用

PMF指標

- [ ] NPS ≥ 30
- [ ] 「なくなったら困る」≥ 60%
- [ ] 月額30万円で「安い」と感じる社数 ≥ 50%

---

15. 見積AI 詳細要件定義（金属加工特化）

> **位置づけ**: Section 1 の見積AIをさらに深掘りした実装仕様。
> **初期ターゲット**: 金属加工（切削・板金・研磨・溶接）。食品・化学は共通基盤の上にPhase 2+で拡張。
> **設計根拠**: 建設積算BPO vs 製造見積BPOの戦略比較分析（下記）に基づく。

15.1 なぜ建設積算ではなく製造見積から攻めるのか

```
建設積算BPOの構造的問題:
  ① 精度要求が厳格: 公共工事は国交省積算基準で±5%以内が法的基準
  ② 図面の空間再構成が必要: 複数図面（平面図+断面図+構造図）を横断した3D理解
     → 燈（企業価値1,000億、エンジニア415名）でも躯体の全自動拾い出しは未解決
  ③ 概算に価値がない: 「±20-30%の概算なら自分でもできる。金払う意味ない」
  ④ 建設積算は積算士が必要だが「ナレッジ吸収→AI移行」戦略で段階的に人件費ゼロに到達可能
     ただし移行に6-10ヶ月かかる。製造見積は初日から人件費ゼロ
  ⑤ 既にBPO市場が確立: ツクノビBPO（700名の積算士）等が先行

製造見積BPOの構造的優位:
  ① 概算精度で価値がある: 概算段階で-25%〜+50%が業界標準。段階的に精度を上げるのが普通
  ② 回答速度=受注率: 「多少高くても素早い回答の工場に発注する」（現場の声）
  ③ 図面が単純: 1部品1図面で完結。空間再構成は不要
  ④ 属人化が深刻: 社長1人が見積を抱えている → 解放されるだけで月25万の価値
  ⑤ 人力BPOが存在しない: AI見積は匠フォース/meviy等が先行するが、BPOとしては空白
  ⑥ 学習ループで精度が上がる: 60%→80%→90%と自律的に改善（スケーラブル）

結論:
  建設積算 = 積算士+AIで即PMF可能。ナレッジ吸収後にAI移行（6-10ヶ月）
  製造見積 = 初日から人件費ゼロ。ただし概算精度から3-6ヶ月の学習が必要
  → どちらから着手するかは「現場へのアクセス」で決まる
```

15.1b 製造見積AI 競合分析（2026年3月時点）

Tier 1: 巨人（正面から戦えない相手）

企業	規模	やっていること	シャチョツーとの関係
CADDi	企業価値700億、累計調達257億	図面データプラットフォーム。類似図面AI検索	エンタープライズ向け。中小には高すぎる
meviy（ミスミ）	東証プライム、売上4,000億	3Dデータ→即時見積→自社で加工・納品	ミスミ自身が加工する。町工場の自社見積には使えない
Xometry	NYSE上場、時価総額$1B+	製造マーケットプレイス	米国中心。日本未進出

Tier 2: 直接競合（最も警戒すべき）

企業	調達額	やっていること	料金	脅威度
匠フォース	累計7.7億（2024年シリーズA 5億）	AI見積+類似図面検索+原価コンサル	月額10万〜	**高い。BPO参入の可能性あり**
SellBOT	非公開	AI見積+類似図面検索+営業支援	月額10万〜	中
Paperless Parts	非公開（米国）	製造業見積ソフト	月額$500-2,000	低（日本未進出）

競合との決定的な違い

```
全社「SaaSツール」を提供 → 社長が自分でソフトを使って見積を作る
シャチョツーは「BPO」     → 社長は図面を送るだけ。見積書が返ってくる

SaaS: 社長にITリテラシーが必要
BPO:  スマホで図面を撮って送るだけ

★ ただし匠フォースがBPOに参入する可能性は高い（5億の資金あり）
→ ウィンドウは6–12ヶ月。先に顧客を取る必要がある
```

シャチョツーの差別化ポイント

```
1. ブレイン統合: 見積AI + 社長の判断基準・ノウハウを構造化
   → 「なぜこの単価にしたか」「この顧客にはいくらで出すか」まで学習
2. ゲノムテンプレート: 業界テンプレートで初日から概算が出せる（コールドスタート解消）
3. BPO包括性: 見積+生産計画+品質管理+在庫+SOP+ISO（8モジュール）
4. ターゲットの違い: 匠フォース=ITリテラシーある工場、シャチョツー=「全部やって」の社長
```

15.2 製造見積の業務フロー（金属加工）

```
顧客からの引き合い
  │
  ▼
Step 1: 図面の受け取り
  ・2D図面（PDF/DXF）or 3Dデータ（STEP/IGES）
  ・口頭/メール仕様（材質、数量、表面処理、納期）
  ・★ 建設と違い「1部品1図面」で完結するケースが大半
  │
  ▼
Step 2: 図面の解読（ベテランの頭の中）
  ・形状の把握: 丸物/角物/板物/溶接構造物
  ・寸法の読み取り: 外径、内径、長さ、穴位置
  ・公差の確認: 一般公差 or 厳しい公差（±0.01mm等）
  ・表面粗さ: Ra3.2, Ra1.6, Ra0.8 等
  ・材質の特性: 被削性、硬度、熱処理の要否
  │
  ▼
Step 3: 加工工程の組み立て（最も属人的な部分）
  ・「この形状 + この材質 + この公差」→ どの工程が必要か？
  ・例: φ30×100mm, S45C, ±0.05mm
    → 丸棒切断 → CNC旋盤（荒加工）→ CNC旋盤（仕上げ）→ 研磨
  ・工程の選択は経験則に大きく依存
  ・同じ部品でも工場の設備構成で工程が変わる
  │
  ▼
Step 4: 工数の見積もり（暗黙知の塊）
  ・段取り時間: 治具セット、プログラム呼び出し、芯出し
    → 類似部品の経験から推定。テンプレート値 ± 調整
  ・サイクルタイム（1個あたりの加工時間）:
    → 形状の複雑さ × 材質の被削性 × 公差の厳しさ で決まる
    → ベテランは「大体8分くらい」と即答できるが、根拠は暗黙知
  ・検査時間: 全数検査 or 抜き取り検査で大きく変わる
  │
  ▼
Step 5: コスト計算
  ・材料費 = 材料寸法（ロス込み）× 材料単価（kg or m）
  ・加工費 = (段取り + サイクルタイム × 数量) / 60 × チャージレート
  ・表面処理 = 外注単価 × 数量（メッキ、アルマイト等）
  ・外注費 = 熱処理、特殊加工等
  ・管理費 = 上記合計 × 管理費率（10-20%）
  ・利益 = 上記合計 × 利益率（10-30%）
  │
  ▼
Step 6: 見積書作成・提出
  ・Excel or PDF で作成
  ・★ 回答速度が受注率を直接左右する
  ・1日以内に返せれば受注率70%+、3日以上で30%以下（業界実感）
```

15.3 見積AIの4層アーキテクチャ（金属加工特化）

```
┌──────────────────────────────────────────────────────────┐
│                 入力: 図面PDF + 仕様情報                  │
│                 （材質・数量・表面処理・納期）              │
└──────────────────────────────────────────────────────────┘
                          │
                          ▼
┌──────────────────────────────────────────────────────────┐
│  Layer 1: 図面解析（建設より大幅に簡単）                   │
│                                                          │
│  入力: 2D部品図（PDF/画像）1枚                             │
│  ・Document AI で寸法・公差・表面粗さをOCR                 │
│  ・正規表現で寸法パターンを構造化抽出                       │
│    "φ30±0.05" → {type: diameter, value: 30, tol: 0.05}│
│    "Ra1.6" → {surface_roughness: 1.6}                    │
│  ・マルチモーダルLLM（Gemini Vision）で形状分類            │
│    → 丸物 / 角物 / 板物 / 溶接構造物                       │
│    → 複雑度スコア（1-5段階）                               │
│                                                          │
│  ★ 建設との決定的な違い:                                   │
│    建設 = 複数図面の空間再構成が必要（未解決問題）          │
│    製造 = 1枚の図面から寸法と形状を読むだけ                 │
│    → OCR + LLM Vision で技術的に解決済みの領域             │
│                                                          │
│  技術: Document AI, Gemini Vision, 正規表現                │
│  精度目標: 寸法読み取り90%+、形状分類85%+                  │
│  開発期間: 3-4週間                                         │
└──────────────────────────────────────────────────────────┘
                          │
                          ▼
┌──────────────────────────────────────────────────────────┐
│  Layer 2: 加工工程推定（コア技術）                         │
│                                                          │
│  入力: Layer1の解析結果 + 材質 + 公差 + 表面処理           │
│  方式: LLMプロンプト + 工程テンプレートDB + 過去実績       │
│                                                          │
│  ■ 方式A: LLM推定（初期）                                  │
│    プロンプト:                                             │
│     「以下の部品の加工工程を推定してください。              │
│      形状: 丸物, φ30×100mm                                 │
│      材質: S45C                                            │
│      公差: ±0.05mm                                         │
│      表面粗さ: Ra1.6                                       │
│      表面処理: 黒染め                                      │
│      → 工程を順に列挙。各工程の設備名・段取り目安も。」     │
│                                                          │
│  ■ 方式B: テンプレートマッチング（精度安定化）             │
│    工程テンプレートDB:                                     │
│    丸物 + 一般公差 → [材料切断, CNC旋盤]                   │
│    丸物 + 精密公差 → [材料切断, CNC旋盤, 研磨]             │
│    角物 + 穴あり → [材料切断, MC, 穴あけ]                  │
│    板物 → [レーザー切断, 曲げ, 溶接]                       │
```

```
│     → 形状×材質×公差で最適テンプレートを選択                │
│     → テンプレート外の特殊工程はLLMで補完                   │
│                                                            │
│  ■ 方式C: 過去実績検索（Level 2以降）                       │
│     類似部品の過去見積を検索（形状・材質・サイズで類似度）│
│     → 過去の実績工程をベースに修正提案                      │
│                                                            │
│  精度目標: 初期70-80%（工程の過不足が1-2工程以内）          │
│  開発期間: 2-3週間                                          │
└────────────────────────────────────────────────────────────┘
                          ▼
┌────────────────────────────────────────────────────────────┐
│  Layer 3: 工数推定（最も難しい部分）                        │
│                                                            │
│  ★ ここが見積AI全体のボトルネック                           │
│  ★ 建設積算の「図面読み取り」に相当する技術的核心           │
│                                                            │
│  ■ 段取り時間の推定                                         │
│     ・設備別デフォルト値（チャージレートマスタに登録）      │
│       CNC旋盤: 20-40分、MC: 30-60分、研磨: 15-30分          │
│     ・部品の複雑度で補正                                    │
│       複雑度1（単純）: デフォルト×0.8                       │
│       複雑度3（標準）: デフォルト×1.0                       │
│       複雑度5（複雑）: デフォルト×1.5                       │
│     ・過去実績があれば実績値で上書き                        │
│     → 精度目標: ±30%（初期）→ ±15%（6ヶ月後）              │
│                                                            │
│  ■ サイクルタイムの推定（最も暗黙知が集中する部分）         │
│     アプローチ1: 加工体積ベース（荒い推定）                 │
│       ・除去体積 = 素材体積 - 完成品体積                    │
│       ・サイクルタイム ≈ 除去体積 / 材料除去率              │
│       ・材料除去率は材質×設備で決まる定数                   │
│       → 精度: ±50%（荒すぎるが初期の出発点）                │
│                                                            │
│     アプローチ2: 特徴量ベース（推奨）                       │
│       ・形状特徴: 外径、長さ、穴数、溝数、ネジ数            │
│       ・材質特徴: 被削性指数（SS400=100%, SUS304=40%）     │
│       ・公差特徴: 一般/精密/超精密                          │
│       ・これらを入力 → 回帰モデル or LLM推定                │
│       → 精度: ±30%（初期）→ ±15%（6ヶ月後）                │
│                                                            │
│     アプローチ3: 類似部品ベース（Level 2以降）              │
│       ・過去に加工した類似部品の実績サイクルタイムを検索    │
│       ・形状類似度 × 材質補正 × 公差補正で推定              │
│       → 精度: ±15-20%（実績データが50件以上あれば）         │
│                                                            │
│  ■ なぜ±30%でも価値があるか                                 │
│     現状: ベテラン1人が30分-2時間かけて見積                 │
│     AI: 3分で概算を出す（±30%精度）                         │
│     → ベテランが確認・修正するのに5-10分                    │
│     → 合計15分で見積完成（従来の1/4-1/8）                   │
│     → 見積回答速度が4-8倍に → 受注率向上                    │
```

```
│                                                          │
│  開発期間: 1-2ヶ月（継続的に精度改善）                    │
└──────────────────────┬──────────────────────────────────┘
                       ▼

┌──────────────────────────────────────────────────────────┐
│  Layer 4: コスト計算 + 見積書生成                         │
│                                                          │
│  ■ コスト計算（決定論的、Section 1.4と同じ）              │
│    材料費 = 素材寸法(ロス率1.1-1.2) × kg単価              │
│    加工費 = Σ(段取り + サイクル×数量) / 60 × チャージ     │
│    表面処理 = 外注単価 × 数量                             │
│    管理費 = 上記合計 × 管理費率                           │
│    利益 = 上記合計 × 利益率                               │
│    見積金額 = 材料費 + 加工費 + 表面処理 + 管理費 + 利益  │
│                                                          │
│  ■ 数量別価格テーブル                                     │
│    1個: 段取り按分が大きい → 高単価                       │
│    10個: 段取り按分1/10 → 中単価                          │
│    100個: 段取り按分1/100 → 低単価                        │
│    → 自動で数量別見積テーブルを生成（営業ツールとして）   │
│                                                          │
│  ■ 受注率分析（Level 2以降）                              │
│    過去の見積→受注/失注データから:                        │
│     「この価格帯の受注率: 70%」                           │
│     「¥5,000下げれば受注率85%に向上」                     │
│    → 利益率と受注率のトレードオフを可視化                 │
│                                                          │
│  ■ 見積書PDF/Excel生成                                    │
│    自社フォーマットのテンプレートに差し込み               │
│    工程別内訳 or 一式のみ（顧客に応じて切替）             │
│                                                          │
│  開発期間: 2-3週間                                        │
└──────────────────────────────────────────────────────────┘
```

15.4 追加データモデル

```
-- 過去見積の実績データ（学習ループ用）
CREATE TABLE mfg_quote_actuals (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  quote_item_id UUID NOT NULL REFERENCES mfg_quote_items(id),
  actual_setup_time_min DECIMAL(8,1),      -- 実績段取り時間
  actual_cycle_time_min DECIMAL(8,1),      -- 実績サイクルタイム
  actual_material_cost BIGINT,             -- 実績材料費
  variance_ratio DECIMAL(5,2),             -- 見積/実績の乖離率
  notes TEXT,
  created_at TIMESTAMPTZ DEFAULT now()
);

-- 工程テンプレートDB（知識ベース）
CREATE TABLE mfg_process_templates (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID,                          -- NULLの場合は共通テンプレート
  template_name TEXT NOT NULL,              -- "丸物_一般公差", "板物_精密" 等
  shape_category TEXT NOT NULL,             -- round / rectangular / plate / welded
  material_group TEXT,                      -- carbon_steel / stainless / aluminum /
copper
  tolerance_level TEXT,                     -- general / precision / ultra_precision
  processes JSONB NOT NULL,                 -- [{process_name, equipment_type,
setup_base_min, cycle_formula, notes}]
  applicable_conditions TEXT,               -- 適用条件の説明
  usage_count INTEGER DEFAULT 0,           -- 使用回数（人気度）
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

-- 図面解析結果（AI解析キャッシュ）
CREATE TABLE mfg_drawing_analyses (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  quote_id UUID NOT NULL REFERENCES mfg_quotes(id),
  file_name TEXT NOT NULL,
  file_path TEXT NOT NULL,
  shape_category TEXT,                      -- round / rectangular / plate / welded
  complexity_score INTEGER,                 -- 1-5
  dimensions JSONB DEFAULT '{}',           -- {outer_diameter, length, holes:
[{x,y,diameter}], grooves: [...]}
  tolerances JSONB DEFAULT '{}',           -- [{dimension, tolerance, type}]
  surface_roughness JSONB DEFAULT '{}',    -- [{surface, Ra}]
  ocr_raw_text TEXT,                        -- OCR生テキスト
  llm_analysis JSONB DEFAULT '{}',         -- LLMの形状分析結果
  confidence FLOAT,                         -- 解析確信度
  created_at TIMESTAMPTZ DEFAULT now()
);
```

```
-- 類似部品検索インデックス（Level 2用）
CREATE TABLE mfg_part_features (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  quote_id UUID NOT NULL REFERENCES mfg_quotes(id),
  shape_category TEXT NOT NULL,
  material TEXT NOT NULL,
  max_dimension DECIMAL(10,2),            -- 最大寸法（mm）
  min_tolerance DECIMAL(8,4),             -- 最小公差（mm）
  hole_count INTEGER DEFAULT 0,
  has_thread BOOLEAN DEFAULT false,
  has_groove BOOLEAN DEFAULT false,
  surface_treatment TEXT,
  feature_vector VECTOR(768),             -- Voyage AI embedding（形状+材質+公差の統合ベクトル）
  created_at TIMESTAMPTZ DEFAULT now()
);

-- RLS
ALTER TABLE mfg_quote_actuals ENABLE ROW LEVEL SECURITY;
ALTER TABLE mfg_process_templates ENABLE ROW LEVEL SECURITY;
ALTER TABLE mfg_drawing_analyses ENABLE ROW LEVEL SECURITY;
ALTER TABLE mfg_part_features ENABLE ROW LEVEL SECURITY;
```

15.5 工程テンプレートDB（初期データ）

```
// brain/genome/data/manufacturing_process_templates.json

{
  "templates": [
    {
      "name": "丸物_一般公差",
      "shape": "round",
      "tolerance": "general",
      "processes": [
        {"name": "材料切断", "equipment": "バンドソー", "setup_min": 5, "cycle_formula":
"length_mm * 0.02"},
        {"name": "CNC旋盤", "equipment": "CNC旋盤", "setup_min": 25, "cycle_formula":
"volume_cm3 * 0.8 + hole_count * 2"}
      ]
    },
    {
      "name": "丸物_精密公差",
      "shape": "round",
      "tolerance": "precision",
      "processes": [
        {"name": "材料切断", "equipment": "バンドソー", "setup_min": 5, "cycle_formula":
"length_mm * 0.02"},
        {"name": "CNC旋盤(荒)", "equipment": "CNC旋盤", "setup_min": 25, "cycle_formula":
"volume_cm3 * 0.8"},
        {"name": "CNC旋盤(仕上)", "equipment": "CNC旋盤", "setup_min": 10,
"cycle_formula": "volume_cm3 * 0.4"},
        {"name": "円筒研磨", "equipment": "円筒研磨盤", "setup_min": 15, "cycle_formula":
"length_mm * 0.05"}
      ]
    },
    {
      "name": "角物_穴あり",
      "shape": "rectangular",
      "tolerance": "general",
      "processes": [
        {"name": "材料切断", "equipment": "バンドソー", "setup_min": 5, "cycle_formula":
"max_dim_mm * 0.03"},
        {"name": "MC(荒)", "equipment": "マシニングセンタ", "setup_min": 40,
"cycle_formula": "volume_cm3 * 1.2"},
        {"name": "MC(仕上)", "equipment": "マシニングセンタ", "setup_min": 15,
"cycle_formula": "volume_cm3 * 0.5 + hole_count * 1.5"}
      ]
    },
    {
      "name": "板金_レーザー曲げ",
      "shape": "plate",
      "tolerance": "general",
      "processes": [
        {"name": "レーザー切断", "equipment": "レーザー加工機", "setup_min": 10,
```

```
"cycle_formula": "perimeter_mm * 0.01"},
        {"name": "曲げ", "equipment": "プレスブレーキ", "setup_min": 15, "cycle_formula":
"bend_count * 0.5"},
        {"name": "溶接", "equipment": "TIG溶接機", "setup_min": 10, "cycle_formula":
"weld_length_mm * 0.03"}
      ]
    },
    {
      "name": "溶接構造物",
      "shape": "welded",
      "tolerance": "general",
      "processes": [
        {"name": "材料切断", "equipment": "バンドソー/レーザー", "setup_min": 10,
"cycle_formula": "part_count * 3"},
        {"name": "仮溶接", "equipment": "半自動溶接機", "setup_min": 20, "cycle_formula":
"weld_point_count * 1"},
        {"name": "本溶接", "equipment": "半自動/TIG溶接機", "setup_min": 15,
"cycle_formula": "weld_length_mm * 0.05"},
        {"name": "歪み取り", "equipment": "プレス/ガス", "setup_min": 10, "cycle_formula":
"weld_length_mm * 0.02"},
        {"name": "仕上げ", "equipment": "グラインダー", "setup_min": 5, "cycle_formula":
"weld_length_mm * 0.01"}
      ]
    }
  ],
  "material_machinability": {
    "SS400": 1.0,
    "S45C": 0.85,
    "S50C": 0.8,
    "SCM435": 0.7,
    "SUS304": 0.4,
    "SUS316": 0.35,
    "A5052": 1.8,
    "A6061": 1.6,
    "A7075": 1.3,
    "C3604": 2.0,
    "Ti-6Al-4V": 0.2
  },
  "complexity_multiplier": {
    "1": 0.7,
    "2": 0.85,
    "3": 1.0,
    "4": 1.3,
    "5": 1.8
  }
}
```

15.6 ヒアリング項目の業種横断テンプレート

```
★ 金属加工に限らず、製造業全般で見積時に聞く項目は共通構造を持つ

共通ヒアリング項目（全製造業で同じ質問をする）:
  1. 何を作るか（図面/仕様/サンプル）
  2. 何で作るか（材料/原料）
  3. いくつ作るか（数量/ロット）
  4. いつまでに（納期）
  5. どの程度の精度で（品質基準/公差/規格）
  6. 仕上げは（表面処理/梱包/検査成績書）

業種別の具体化:
  | 共通項目 | 金属加工 | 食品加工 | 化学 | 電子部品 |
  |---|---|---|---|---|
  | 仕様 | 2D/3D図面 | レシピ/配合表 | 反応条件/SDS | 回路図/BOM |
  | 材料 | SS400/SUS304 | 原材料/添加物 | 原料/溶媒 | 基板/IC/受動素子 |
  | 数量 | ロット数 | 生産量(kg/L) | バッチサイズ | ロット数 |
  | 精度 | 公差/表面粗さ | 食品衛生法 | 純度/含有量 | 電気特性 |
  | 仕上げ | メッキ/塗装 | 包装/ラベル | 充填/梱包 | はんだ/実装 |

→ ゲノムテンプレート（brain/genome/）で業種ごとに具体化
→ ブレインのナレッジ入力（Q&A）で「見積時のヒアリング知識」を構造化
→ 見積AIは構造化されたナレッジを参照して工程推定・工数推定を行う
```

15.6b ブレインによるナレッジ蓄積とスケール

```
製造見積の知識を3層に分解:

Layer 1: ヒアリング（何を聞くか）→ 全業種100%共通
   材料・数量・工程・納期・品質・仕上げ
   → プラットフォーム側で1回作れば全社に適用

Layer 2: ビジネスロジック（どう計算するか）→ 全業種90%共通
   材料費 + 加工費 + 外注費 + 管理費 + 利益
   → 計算エンジンは共通

Layer 3: ドメイン知識（具体的にいくらか）→ 各社固有
   チャージレート、工程テンプレート、材料単価
   → ブレインに蓄積して同業種内で転用

N社目のオンボーディングコスト推移:
   同業種1社目:   数日〜1週間（ゼロから構築）
   同業種2社目:   1–2日（1社目のナレッジをテンプレート提示、差分のみ修正）
   同業種5社目:   半日（ほぼテンプレート確認のみ）
   異業種1社目:   数日（ゲノムテンプレートで初期値提供、固有値のみ修正）

★ Layer 1–2は「作れば終わり」。Layer 3はブレイン+ゲノムでスケールする
   ただし新業種参入時はゲノムテンプレートの新規作成が必要（建設より重い）
```

15.7 学習ループ（精度改善の仕組み）

```
見積AI最大の特徴: 「使うほど賢くなる」フライホイール

┌──────────────────────────────────────────┐
│  1. AIが見積を生成（精度60-70%）              │
│     ├─ 工程推定                              │
│     ├─ 工数推定（段取り + サイクルタイム）     │
│     └─ コスト計算                            │
│                                              │
│  2. ベテランが確認・修正（5-10分）            │
│     ├─ 「この工程は不要」→ 削除              │
│     ├─ 「段取りは40分じゃなく25分」→ 修正     │
│     └─ 「サイクルは8分じゃなく12分」→ 修正    │
│     → ★ この修正データが次の学習データになる   │
│                                              │
│  3. 受注/失注の記録                          │
│     ├─ 受注 → 金額の妥当性が証明              │
│     └─ 失注 → 理由記録（価格/納期/品質）      │
│                                              │
│  4. 実績データの記録（加工完了後）            │
│     ├─ 実績段取り時間 vs 見積段取り時間       │
│     ├─ 実績サイクルタイム vs 見積サイクルタイム│
│     └─ 実績材料使用量 vs 見積材料量           │
│     → 乖離率を算出 → 次回の推定を自動補正     │
│                                              │
│  5. 月次再訓練                               │
│     ├─ 修正データ + 実績データを統合          │
│     ├─ 工程テンプレートの精度を更新           │
│     ├─ サイクルタイム推定モデルを再訓練       │
│     └─ 類似部品検索インデックスを更新         │
└──────────────────────────────────────────┘

精度の推移（予測）:
  Month 0:   60-70%（テンプレート + LLM推定のみ）
  Month 1-2: 70-75%（ベテランの修正データ蓄積）
  Month 3-4: 75-80%（実績データとの照合が回り始める）
  Month 6:   80-85%（類似部品検索が効き始める）
  Month 12:  85-90%（十分な実績データ。ベテラン並み）

★ 建設積算との決定的な違い:
  建設: 精度改善に「図面解析技術の進化」が必要（外部要因）
  製造: 精度改善に「使用量の増加」だけが必要（内部要因）
  → 顧客が使うほど賢くなる → 解約しにくい → LTV最大化
```

15.8 技術スタック

```
# Layer 1: 図面解析
google-cloud-documentai       # OCR（寸法・公差・表面粗さ読み取り）
google-genai                  # Gemini Vision（形状分類・複雑度判定）
opencv-python                 # 図面前処理（ノイズ除去・二値化）

# Layer 2: 工程推定
# → LLMプロンプト + 工程テンプレートDB（新規コード）
# → brain/genome/data/manufacturing_process_templates.json

# Layer 3: 工数推定
scikit-learn                  # 回帰モデル（サイクルタイム推定）Level 2以降
voyageai                      # 類似部品検索用embedding（既存）

# Layer 4: コスト計算・見積書
openpyxl                      # Excel見積書生成
reportlab                     # PDF見積書生成（Phase 2+）

# インフラ（建設版より軽い）
# GPU: 不要（物体検出モデルの訓練は不要。LLM APIで完結）
# LLM API: Gemini 2.5 Flash（月¥1-3万）
# OCR API: Document AI（月¥1-2万）
# → 月額インフラコスト: ¥3-5万（建設版の1/3）
```

15.9 開発ロードマップ

```
Phase A（3週間）: Level 1 — 工程推定AI
  ├─ Week 1: 図面解析パイプライン（Document AI + Gemini Vision）
  ├─ Week 2: 工程テンプレートDB + LLM工程推定
  ├─ Week 3: コスト計算 + 見積書Excel生成
  └─ 成果: 図面→工程→見積書の基本パイプライン

Phase B（3週間）: UI + 確認フロー
  ├─ Week 4: 見積作成UI（図面アップロード→AI解析→工程表示）
  ├─ Week 5: 工程・工数の編集UI + チャージレート管理
  ├─ Week 6: 見積一覧 + 受注/失注記録 + 見積書出力
  └─ 成果: パイロット企業に投入可能なMVP

Phase C（2週間）: 学習ループ
  ├─ Week 7: 修正データの蓄積 + 実績データ入力UI
  ├─ Week 8: 実績 vs 見積の乖離分析レポート
  └─ 成果: 「使うほど賢くなる」基盤

Phase D（Phase 2+）: Level 2 — 実績学習型見積
  ├─ 類似部品検索（Voyage AI embedding）
  ├─ サイクルタイム回帰モデル（scikit-learn）
  ├─ 受注率分析
  └─ 数量別価格テーブル自動生成

合計 Phase A-C: 8週間（建設の図面自動拾い出しAI 10ヶ月 vs 製造見積AI 2ヶ月）
```

15.10 建設積算AIとの比較まとめ

比較軸	建設積算AI（Section 17）	製造見積AI（本セクション）
図面解析の難度	極めて高い（複数図面の空間再構成）	**低い（1枚の部品図から寸法読み取り）**
GPU/専用モデル	必要（YOLO訓練 + 推論GPU）	**不要（LLM APIで完結）**
月額インフラコスト	¥10-15万	**¥3-5万**
開発期間	10ヶ月	**2ヶ月**
初期精度	85-90%（部材検出）	**60-70%（工数推定）**
精度改善方法	訓練データ追加（外部要因）	**使用量増加（内部要因）**
精度要求	±5%（公共工事は法的基準）	**±20-30%（概算段階で許容）**
PMFまでの期間	積算士+AIなら1-2ヶ月 / 図面AI完成は10ヶ月	**2-3ヶ月**
競合	燈(1,000億)+ツクノビBPO。ただし中小向けBPOは空白	**SaaSはレッドオーシャン（匠フォース7.7億/CADDi 700億/SellBOT）。BPOは空白だがウィンドウ6-12ヶ月**
スケーラビリティ	中（積算士は段階的にAI移行可能。6-10ヶ月で人件費ゼロ）	**高い（AI+確認で完結。ただし各社セットアップが必要）**

15.11 成功基準

```
Phase A 完了基準（3週間目）:
  - [ ] 図面PDF → 形状分類 + 寸法抽出が動作
  - [ ] 工程テンプレートから工程リスト自動生成
  - [ ] コスト計算 → 見積書Excel出力
  - [ ] パイロット1社の実図面で「概算として使える」と評価

Phase B 完了基準（6週間目）:
  - [ ] 見積作成UIが稼働
  - [ ] 工程・工数の人間修正が可能
  - [ ] パイロット3社で実業務テスト開始
  - [ ] 見積作成時間が50%以上短縮

Phase C 完了基準（8週間目）:
  - [ ] 修正データが蓄積される仕組みが稼働
  - [ ] 実績データ入力UIが稼働
  - [ ] 乖離率レポートが自動生成

6ヶ月運用後:
  - [ ] 工数推定精度 ±20%以内
  - [ ] 見積作成時間: 平均30分→10分
  - [ ] 受注率が10%以上向上（測定可能な場合）
  - [ ] 「なくなったら困る」≥ 60%
```